
Codificación de Canal: Detección y Corrección de Errores

Por: Ing. Cristopher Mc Guinness

El canal puede cambiar los bits que enviamos

- Hasta ahora supusimos que el receptor recupera los bits que el transmisor puso en el canal.
- El ruido térmico, el desvanecimiento (atenuación variable) y el crosstalk (interferencia entre pares) pueden invertir bits.
- Subir la potencia ayuda, pero la potencia **no siempre está disponible ni es barata**.
- La codificación de canal ofrece otra moneda: **gastar bits en lugar de Watts**.

De la semana anterior a la pregunta de hoy

- La **codificación de fuente** quita redundancia para representar información con menos bits.
- La **codificación de canal** añade redundancia calculada para proteger esos bits.
- No se contradicen: primero comprimimos, después protegemos el resultado.
- Hoy seguiremos un mensaje de cuatro bits: **1001**. Veremos cómo enviarlo, comprobarlo y recuperarlo.

La idea: añadir redundancia de forma estructurada

- Un código de bloque (n, k) convierte k bits de información en una palabra válida de n bits: añade $n - k$ bits calculados.
- Esa redundancia **no lleva información nueva**: lleva información *sobre* la información.
- El receptor la usa para comprobar coherencia y, si procede, reparar.
- X : es el mensaje.
- C : la palabra código enviada.
- R : el bloque recibido.
- E : marca con 1 cada bit invertido.
- Son **vectores fila**: listas ordenadas de bits. x_i, c_i, r_i son sus bits, numerados desde 1, un sombrero indica una estimación del receptor.
- El **modulador** convierte bits en señales físicas, el **demodulador** obtiene de ellas muestras o decisiones sobre los bits.

Detectar y corregir no son lo mismo

- **Detectar** es comprobar que la palabra recibida no pertenece a los códigos válidos. Es barato.
- **Corregir** es además decidir cuál era la palabra enviada. Cuesta mucha más redundancia.
- Detectar sin corregir permite **descartar, avisar o pedir retransmisión**, si hay retorno.
- **ARQ**, petición automática de repetición (*Automatic Repeat reQuest*), pide otra copia por un canal de retorno.
- **FEC**, corrección hacia adelante (*Forward Error Correction*), intenta reparar con la redundancia ya recibida, sin pedir otra copia.

El recorrido: una pregunta por etapa

Pregunta

- ¿Cómo advertir que algo cambió?
- ¿Cuántos cambios podemos reparar?
- ¿Cómo hacerlo sin tablas gigantes?
- ¿Compensa el coste?

Herramienta que construiremos

- XOR: suma binaria, paridad: comprobar un total par de unos
- Distancia mínima: separación en bits entre palabras válidas
- G construye, H comprueba, el síndrome reúne los resultados de esas comprobaciones
- Error de bloque, energía y banda

- Después: decisión blanda o “soft”(usar también la confianza de las muestras), otras familias y ARQ como complementos.
- Cada herramienta se acompaña de un ejemplo antes de pasar a la siguiente.

Tres situaciones donde no se puede simplemente subir la potencia

- **Canal con desvanecimiento:** la atenuación varía con el tiempo, la potencia media no evita los desvanecimientos profundos.
- **Enlace por satélite:** la potencia de la sonda está fijada por el panel solar, no hay margen para subirla.
- **Crosstalk:** subir la potencia **incrementa también la interferencia** sobre los pares vecinos.

En los tres casos el cuello de botella no es la potencia, sino la **fiabilidad por unidad de energía**.

Antes de las matrices: XOR en cuatro cuentas

- **XOR**, o suma módulo 2, devuelve el resto de dividir la suma entre 2. Lo escribimos $\oplus$.

$$0 \oplus 0 = 0 \quad 0 \oplus 1 = 1 \quad 1 \oplus 0 = 1 \quad 1 \oplus 1 = 0$$

- No hay acarreo: dos unos se cancelan.
- En vectores se aplica posición por posición: $1001 \oplus 0101 = 1100$.
- **Paridad par** significa un total par de unos: mensaje 101, bit de paridad 0, palabra 1010.
- Si cambia el último bit, llega 1011: la suma de sus bits es 1 y **se detecta**.

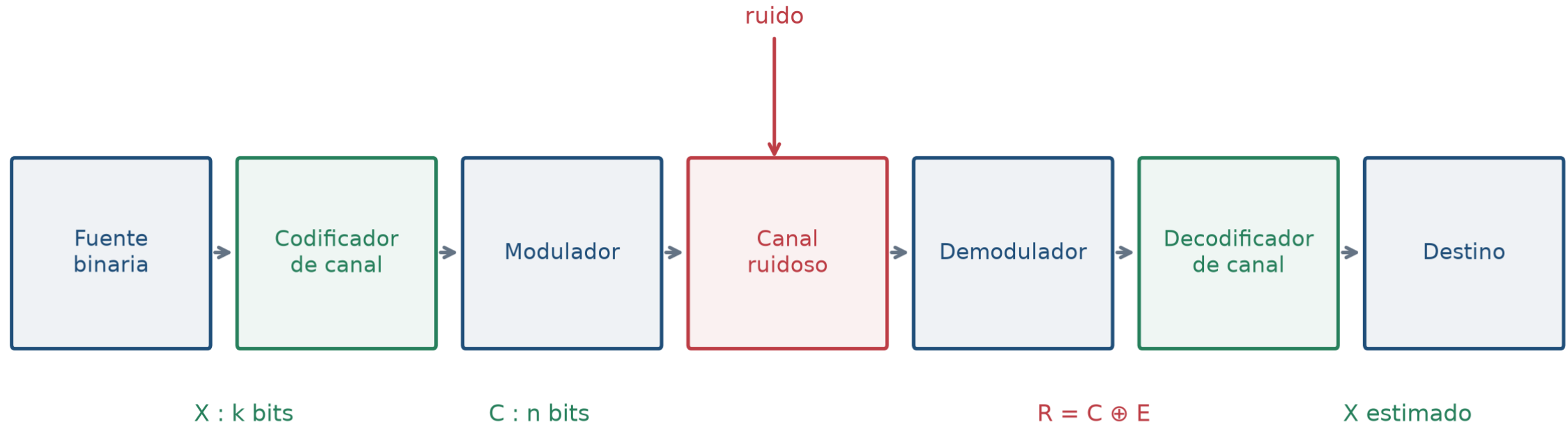
El precio y el símbolo que lo mide

- El **throughput** cuenta bits de información por segundo, el **ancho de banda** es el intervalo de frecuencias ocupado, en Hz.
- La fracción útil de los bits transmitidos se llama **tasa de código**:

$$R_c = \frac{k}{n} < 1$$

- R_c es la **tasa de código**: la fracción del flujo que lleva información útil.
- Su inverso $1/R_c$ es el **factor de expansión**: cuánto crece el flujo transmitido.
- Usaremos el código llamado **Hamming (7,4)**: $R_c = 4/7 \approx 0,571$ y el flujo crece un 75 %. Más adelante construiremos sus comprobaciones.
- Menos tasa deja espacio para más redundancia, pero **no garantiza** más protección: importa cómo se construya el código.
- El coste es **más ancho de banda o menos throughput**.

La cadena de corrección de errores hacia adelante (FEC)



El codificador anade $n - k$ bits de paridad: tasa $R_c = k/n < 1$

El decodificador de canal es el espejo del codificador

El codificador convierte X de k bits en C de n bits. El canal suma un patrón de error E . El decodificador recibe R y estima cuál era X .

Paridad simple: un solo bit de comprobación

$$c_n = x_1 \oplus x_2 \oplus \cdots \oplus x_{n-1}$$

- Se elige el bit de paridad para que el total de unos sea **par** (paridad par).
- Código $(n, n - 1)$ con $R_c = (n - 1)/n$: redundancia mínima.
- El receptor suma módulo 2 todo el bloque: si da 1, hubo error.

Ocho palabras válidas, no dieciséis

x_1	x_2	x_3	$c_4 = x_1 \oplus x_2 \oplus x_3$	Palabra C
0	0	0	0	0000
0	0	1	1	0011
0	1	0	1	0101
0	1	1	0	0110
1	0	0	1	1001
1	0	1	0	1010
1	1	0	0	1100
1	1	1	1	1111

- Son las 2^3 palabras de paridad par de longitud 4.
- Si llega una de las otras ocho, sabemos que hubo error.
- Si llega una válida, **no sabemos aún** si fue la que se transmitió.

Ejemplo 10.2: qué demuestra la paridad

Operación módulo 2

$$0011 \oplus 1100 = 1111$$

$$1010 \oplus 1010 = 0000$$

$$1001 \oplus 0001 = 1000$$

$$1010 \oplus 0011 = 1001$$

Interpretación

- La suma de palabras válidas sigue siendo válida
 - Toda palabra se cancela consigo misma
 - Un error sale del conjunto válido
 - Dos errores pueden llevar a otra palabra válida
- La primera propiedad se llama **linealidad**, pronto permitirá usar matrices.

Peso y distancia

- **Peso** cuenta unos, **distancia de Hamming** cuenta posiciones distintas. Aquí C_i y C_j son dos palabras completas, no dos bits.

$$w_H(C) = \text{número de unos de } C$$

$$d_H(C_i, C_j) = \text{número de posiciones en que difieren}$$

- En aritmética módulo 2 las dos ideas se enlazan:

$$d_H(C_i, C_j) = w_H(C_i \oplus C_j)$$

- La distancia cuenta **cuántos bits habría que invertir** para pasar de una palabra a la otra.
- Por ejemplo, 0011 y 0101 difieren en dos posiciones, su XOR es 0110, de peso 2.

La distancia mínima del código

- $d_{\min}$ es la **menor distancia de Hamming entre dos palabras válidas distintas**, no entre mensaje y recibido.

$$d_{\min} = \min d_H(C_i, C_j) \text{ con } i \neq j$$

- Es la **distancia del par más cercano**: el eslabón débil del código.
- En un código **lineal** hay un atajo decisivo:

$$d_{\min} = \min w_H(C) \text{ con } C \neq 0$$

- $\binom{2^k}{2} = \frac{2^k(2^k-1)}{2}$ cuenta las maneras de elegir dos palabras distintas entre 2^k . No hay que comparar esos pares: basta pesar las $2^k - 1$ palabras no nulas.
- Para $n = 4$ el código es $(4,3)$ y su $d_{\min}$ vale **2**.

Cota de detección

$$\text{errores detectables} = d_{\min} - 1$$

- Con menos de $d_{\min}$ errores es **imposible** caer sobre otra palabra código válida.
- El bloque recibido queda "entre" palabras y el receptor sabe que algo falló.
- Con exactamente $d_{\min}$ errores ya se puede aterrizar en otra palabra: error indetectable.
- Paridad simple($n, n-1$): $d_{\min} = 2$, detecta 1.
- Hamming (7,4): $d_{\min} = 3$, detecta 2.

Cota de corrección

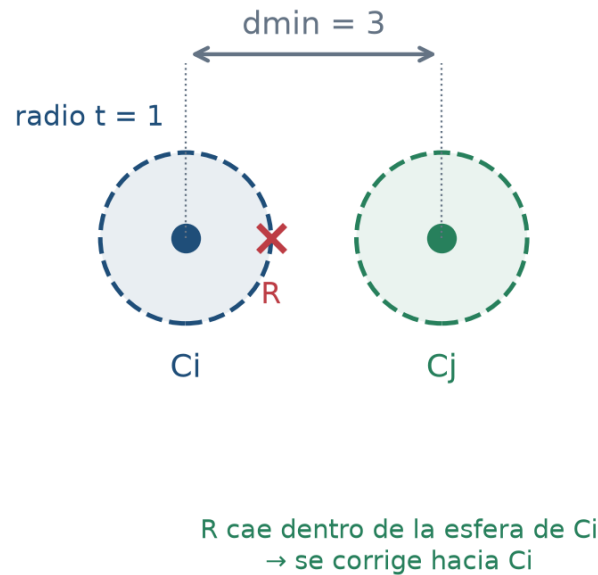
- t es el número de errores que se garantiza corregir. Los corchetes $\lfloor \cdot \rfloor$ significan **redondear hacia abajo**.

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

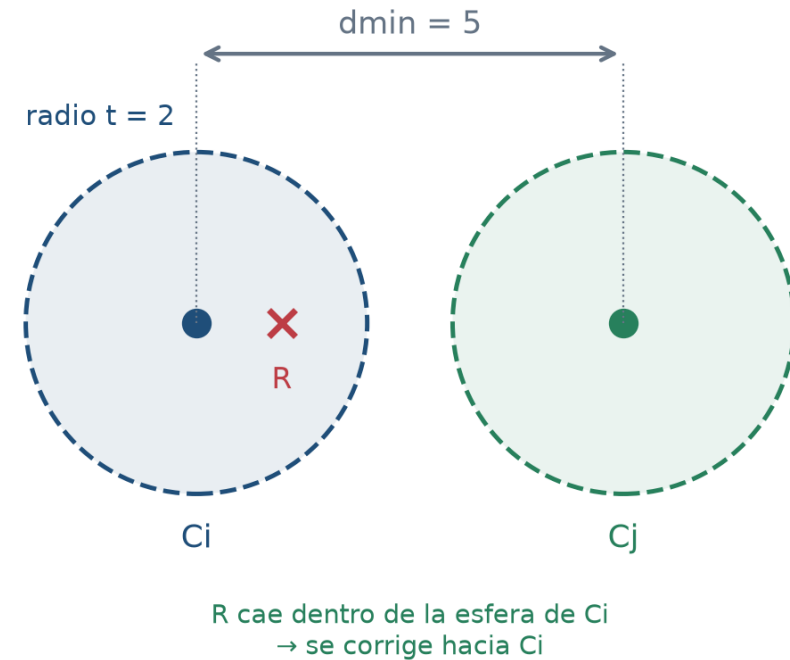
- Para **corregir** no basta con saber que hubo error: hay que decidir cuál era la palabra original.
- El receptor elige la palabra código **más cercana** al bloque recibido.
- Esa decisión es correcta mientras el error mueva la palabra menos de la mitad de $d_{\min}$.
- Corregir exige **más separación** que detectar: para corregir t errores se necesita al menos una distancia $2t + 1$.
- Una **esfera de radio t** reúne todos los bloques que difieren de su palabra central en, como máximo, t bits.

La geometría de las esferas de decodificación

$d_{\min} = 3 \rightarrow$ detecta 2, corrige $t = 1$



$d_{\min} = 5 \rightarrow$ detecta 4, corrige $t = 2$



Cada palabra código reserva una esfera de radio t a su alrededor

Para que las esferas no se solapen hace falta $2t < d_{\min}$. Todo lo que caiga dentro de una esfera se corrige hacia su centro.

La estrategia mixta: repartir la distancia

- Ya conocemos las cotas. Ahora elegimos **cuándo corregir y cuándo pedir otra copia**, sin cambiar el código.
- e_d es el numero de errores detectables. Por encima de t se **avisa sin corregir**.

$$t + e_d \leq d_{\min} - 1, \quad e_d \geq t$$

- Para un código con $d_{\min} = 5$, distinto del Hamming (7,4), caben estas políticas:

Política para $d_{\min} = 5$

- $t = 2, e_d = 2$
- $t = 1, e_d = 3$
- $t = 0, e_d = 4$

Garantía

- Corrige hasta dos, no garantiza avisar de tres
- Corrige uno, detecta dos o tres sin corregir
- Solo detecta, hasta cuatro, útil para ARQ

- Con $t = 1$ corregimos solo a distancia 0 o 1. Si ninguna palabra está así de cerca, **avisamos en lugar de adivinar**.

Cómo contar errores de bits independientes

$$\binom{n}{j} = \frac{n!}{j! (n-j)!}$$

- P'_e es la **probabilidad de error de un bit de canal**, antes de corregir, $1 - P'_e$ es la de acertarlo.
- Suponemos igual probabilidad en cada posición y **errores independientes**: saber qué pasó en un bit no cambia la probabilidad de los demás.

- Para un bloque de n bits, j cuenta cuántos se invirtieron:

$$\text{Prob}\{j \text{ errores en } n \text{ bits}\} = \binom{n}{j} (P'_e)^j (1 - P'_e)^{n-j}$$

- Para un patrón concreto, multiplicamos j probabilidades de error y $n - j$ probabilidades de acierto.
- El coeficiente $\binom{n}{j}$ cuenta cuántos patrones diferentes tienen exactamente j errores. Los veremos de inmediato.
- El signo Σ indica **sumar los casos indicados**: los pares no nulos para paridad, o los que superan t para un corrector.
- $P_{\text{no det}}$ cuenta bloques erróneos aceptados sin aviso. No es lo mismo que un error de bit ni que un fallo tras corregir.
- **BER** es la proporción de bits incorrectos. Una probabilidad de bloque cuenta bloques, aunque cada uno tenga varios bits incorrectos.

Por qué aparece el coeficiente binomial

- Cuatro bits y **exactamente dos errores**. Un 1 marca un bit invertido.

Posiciones que fallan	Patrón E	Probabilidad de ese patrón
1 y 2	1100	$(P'_e)^2(1 - P'_e)^2$
1 y 3	1010	$(P'_e)^2(1 - P'_e)^2$
1 y 4	1001	$(P'_e)^2(1 - P'_e)^2$
2 y 3	0110	$(P'_e)^2(1 - P'_e)^2$
2 y 4	0101	$(P'_e)^2(1 - P'_e)^2$
3 y 4	0011	$(P'_e)^2(1 - P'_e)^2$

- Los seis casos no pueden ocurrir a la vez. **Sumar sus probabilidades equivale a multiplicar por 6.**

La misma cuenta escrita con factoriales

- El factorial multiplica los enteros hasta el número indicado. Por definición, $0! = 1$.

$$n! = n(n-1) \cdots 2 \cdot 1 \quad \binom{n}{j} = \frac{n!}{j!(n-j)!}$$

$$\binom{4}{2} = \frac{4!}{2!2!} = \frac{4 \cdot 3 \cdot 2 \cdot 1}{(2 \cdot 1)(2 \cdot 1)} = 6$$

$$\text{Prob}\{2 \text{ errores}\} = 6(0,01)^2(0,99)^2 = 0,00058806$$

- Hay 1 % de probabilidad de error **por bit**, pero 0,058806 % de probabilidad de **exactamente dos errores en el bloque**.
- Dividimos entre los factoriales porque el orden dentro de cada grupo no crea un patrón nuevo.

Probabilidad de error de bloque

- P_b es la probabilidad de que el **bloque quede incorrecto después de decodificar**. No es una probabilidad por bit.

$$P_b \leq \sum_{j=t+1}^n \binom{n}{j} (P'_e)^j (1 - P'_e)^{n-j}$$

- La suma reúne los casos con **más de t errores**, con errores de bit independientes.
- Un código general podría corregir algunos patrones de peso mayor. Por eso la expresión es una cota, no siempre una igualdad.
- En Hamming (7,4), con el corrector indicado, es igualdad.

El punto ciego de la paridad simple

- $d_{\min} = 2$ permite **detectar 1 error y corregir ninguno**.
- Detecta cualquier número **impar** de errores, los pares dejan la paridad intacta.
- Con errores independientes, $P'_e = 10^{-4}$ y $n = 8$:

$$P_{\text{no det}} = \sum_{j=2,4,6,8} \binom{8}{j} (P'_e)^j (1 - P'_e)^{8-j} = 2,7983 \times 10^{-7}$$

- Pequeña, pero **no nula**: es el precio de un único bit de comprobación.

Paridad de cuatro bits: la cuenta completa

- Canal con $P'_e = 1 \% = 0,01$. La paridad (4,3) deja pasar exactamente **dos o cuatro errores**.

$$P_{\text{no det}} = \binom{4}{2} (P'_e)^2 (1 - P'_e)^2 + \binom{4}{4} (P'_e)^4$$

$$P_{\text{no det}} = \frac{4!}{2! 2!} (0,01)^2 (0,99)^2 + \frac{4!}{4! 0!} (0,01)^4$$

$$P_{\text{no det}} = 6(0,01)^2 (0,99)^2 + (0,01)^4 = 0,00058806 + 0,00000001$$

- Resultado: **0,00058807**, aproximadamente $5,88 \times 10^{-4}$. Es error no detectado, no BER.

Diez bits: desarrollar todos los casos con más de dos errores

- Sumamos $j = 3, 4, 5, 6, 7, 8, 9$ y 10 , con probabilidad por bit de $0,01$.

$$P_{>2} = \sum_{j=3}^{10} \frac{10!}{j! (10-j)!} (0,01)^j (0,99)^{10-j}$$

$$\begin{aligned} P_{>2} = & 120(0,01)^3(0,99)^7 + 210(0,01)^4(0,99)^6 + 252(0,01)^5(0,99)^5 \\ & + 210(0,01)^6(0,99)^4 + 120(0,01)^7(0,99)^3 + 45(0,01)^8(0,99)^2 \\ & + 10(0,01)^9(0,99) + (0,01)^{10} \approx 1,13849 \times 10^{-4} \end{aligned}$$

- Esto cuenta errores del canal. **No demuestra que exista un código (10,5) capaz de corregir dos errores.**

Código rectangular: paridad en dos direcciones

k = 20 bits utiles

1	1	1	1	1	1
1	1	0	0	0	0
0	1	1	0	0	0
1	0	1	0	0	0
1	1	1	1	1	1

4 paridades de fila

5 paridades de columna

esquina

$n - k = 4 + 5 + 1 = 10$

codigo (30, 20)

$R_c = 20/30 = 2/3$

$d_{min} = 4$

corrige 1 error:
la fila y la columna
que fallan lo sitúan

La intersección de la fila y la columna que fallan localiza el error

Con paridad solo en una dirección se detecta, con paridad en las dos, se **corrige** un error simple.

Las cuentas del código producto

$$n - k = 4 \text{ filas} + 5 \text{ columnas} + 1 \text{ esquina} = 10$$

$$(n, k) = (30, 20)$$

$$R_c = \frac{20}{30} = \frac{2}{3}$$

$$d_{\min} = 4$$

- $d_{\min} = 4$ es el producto de las distancias de los dos códigos de paridad componentes, 2×2 .
- Con una política de radio 1: corrige un error y **señala sin corregir** los dobles.
- Saltamos de "detectar uno" a "corregir uno" **sin salir de la paridad**.

Contraejemplo: tres bits de paridad no bastan

- Un código (7,4) alternativo, aparentemente razonable:

$$c_5 = x_1 \oplus x_2 \oplus x_3 \oplus x_4, \quad c_6 = x_1 \oplus x_2, \quad c_7 = x_3 \oplus x_4$$

- Tiene el mismo n , el mismo k y la misma tasa que el Hamming.
- Pero la palabra 1100000 es válida y tiene **peso 2**, luego $d_{\min} = 2$.
- Detecta cualquier error simple, pero **no garantiza corregirlos todos**.

Qué significa que un código sea lineal

Un código de bloque es **lineal** si la suma módulo 2 de dos palabras código cualesquiera es también una palabra código.

- Consecuencia inmediata: la palabra cero aparece porque cualquier palabra sumada consigo misma se cancela.
- Un **subespacio vectorial** es un conjunto cerrado al XOR dentro del espacio de n bits.
- Es suficiente elegir k palabras código como “bloques de construcción” para obtener todas las palabras válidas mediante XOR.
- De 2^n bloques posibles, solo 2^k son válidos: los demás muestran errores.
- La linealidad es lo que permite el atajo $d_{\min} = \min w_H(C)$ y la decodificación por síndrome.

Por qué no sirve una tabla de consulta

- Una tabla de consulta guarda una salida por cada mensaje posible. Este es un extracto del código (7,4):

Mensaje X	Palabra C	Qué se guarda
0000	0000000	7 bits de salida
0001	0001011	7 bits de salida
0010	0010110	7 bits de salida
1001	1001110	7 bits de salida
1111	1111111	7 bits de salida

- El (7,4) tiene 16 filas. El (127,92) necesitaría $2^{92} \approx 5 \times 10^{27}$ filas.
- Solo sus salidas ocuparían aproximadamente **$7,86 \times 10^{28}$ bytes**. Guardar G requiere solo **11 684 bits**.
- La alternativa será describir las palabras mediante una **matriz**, una tabla rectangular de bits, en vez de almacenarlas todas.

Codificar es multiplicar por una matriz

- G es la **matriz generadora**, de tamaño $k \times n$: k filas y n columnas, con aritmética módulo 2.

$$C = XG$$

- Sus k filas son las palabras código de los mensajes $100 \dots 0, 010 \dots 0, \dots$
- Una **combinación lineal** aquí es el XOR de las filas seleccionadas por los unos del mensaje, eso calcula XG .
- Toda palabra código es una combinación lineal de las filas de G .
- Codificar pasa de ser una búsqueda gigante a unas pocas operaciones XOR.

Ver G en acción: seleccionar filas y hacer XOR

G

	c1	c2	c3	c4	c5	c6	c7
x1	1	0	0	0	1	0	1
x2	0	1	0	0	1	1	1
x3	0	0	1	0	1	1	0
x4	0	0	0	1	0	1	1

I4 copia x1, x2, x3, x4

P

X = 1 0 0 1
Elegir filas 1 y 4

fila 1: 1 0 0 0 1 0 1

fila 4: 0 0 0 1 0 1 1

XOR: 1 0 0 1 1 1 0

$C = XG = 1001110$. Toda la suma es módulo 2.

Los unos de X seleccionan las filas que se suman

Para 1001, hacemos XOR de las filas 1 y 4. I4 es la identidad, unos en la diagonal y ceros fuera. Copia el mensaje. P calcula las tres paridades.

Forma sistemática: el mensaje queda a la vista

- I_k es la **matriz identidad**: unos en la diagonal y ceros fuera, copia el mensaje. P reúne las reglas que calculan la paridad.
- La barra vertical significa colocar dos bloques de columnas **uno junto al otro**:

$$G = [I_k | P] \implies C = [x_1 \dots x_k | c_{k+1} \dots c_n]$$

- Los primeros k bits de la palabra código **son** el mensaje, sin transformar.
- Los $n - k$ restantes son las comprobaciones que genera el bloque P .
- Ventaja práctica: si no hubo error, **extraer el mensaje es truncar**.
- Todo código lineal admite una forma sistemática equivalente: intercambiar filas, hacer XOR entre ellas y, si hace falta, permutar (reordenar) posiciones.
- La linealidad ofrece la alternativa: en vez de 2^k palabras, guardar $k \times n$ bits.
- Si G es sistemática, basta guardar P : $k(n - k)$ bits.

Las tres ecuaciones de paridad

$$c_5 = x_1 \oplus x_2 \oplus x_3$$

$$c_6 = x_2 \oplus x_3 \oplus x_4$$

$$c_7 = x_1 \oplus x_2 \oplus x_4$$

- Cada bit de paridad cubre **un subconjunto distinto** de tres bits de mensaje.
- Los subconjuntos se solapan: por eso cada error **simple** produce una firma única.
- Son exactamente las columnas del bloque P , leídas hacia abajo.

Nuestro mensaje 1001, ahora codificado

- Sustituimos $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1$:

$$c_1 = 1$$

$$c_2 = 0$$

$$c_3 = 0$$

$$c_4 = 1$$

$$c_5 = 1 \oplus 0 \oplus 0 = 1$$

$$c_6 = 0 \oplus 0 \oplus 1 = 1$$

$$c_7 = 1 \oplus 0 \oplus 1 = 0$$

$$C = 1001110$$

- La misma cuenta con G: sumamos sus filas 1 y 4, porque son los unos de X.
- Enviamos **1001 + 110**: cuatro bits de información y tres de paridad.

La matriz de comprobación de paridad

- H es la **matriz de comprobación de paridad**. Se construye poniendo P arriba y la identidad debajo.
- 0 indica una matriz o un vector de ceros del tamaño que corresponda.
- Tiene n filas y $n - k$ columnas. Cada columna selecciona los bits de una comprobación.

$$GH = 0 \implies CH = XGH = 0$$

- H codifica las mismas ecuaciones de paridad, pero como **prueba** en lugar de como receta.
- $CH = 0$ es la **definición operativa** de palabra código válida.
- G construye, H verifica. Son las dos caras del mismo subespacio.

Ver H en acción: comprobar una palabra válida

Columnas = comprobaciones

	1	2	3	
c1	1	0	1	] 13
c2	1	1	1	
c3	1	1	0	
c4	0	1	1	
c5	1	0	0	
c6	0	1	0	
c7	0	0	1	
H				

Comprobar C = 1001110

$$1: c1 \oplus c2 \oplus c3 \oplus c5 = 1 \oplus 0 \oplus 0 \oplus 1 = 0$$

$$2: c2 \oplus c3 \oplus c4 \oplus c6 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$3: c1 \oplus c2 \oplus c4 \oplus c7 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

CH = (0 0 0): cumple las tres paridades

Las dimensiones explican por qué funciona

Producto		Dimensiones del resultado
•	XG	• $(1 \times k)(k \times n) = 1 \times n$
•	RH	• $(1 \times n)(n \times (n - k)) = 1 \times (n - k)$

$$GH = I_k P \oplus P I_{n-k} = P \oplus P = 0$$

- La identidad copia P, la segunda copia se cancela con la primera.
- Así, todas las palabras construidas por G satisfacen las comprobaciones de H.
- **Trasponer** una matriz intercambia sus filas y columnas. No es necesario trasponer P en la convención de H adoptada aquí.

Las matrices del código de Hamming (7,4)

4 filas × 7 columnas

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \quad P$$

$c1 = x1 \quad c2 = x2 \quad c3 = x3 \quad c4 = x4$

$$c5 = x1 \oplus x2 \oplus x3$$

$$c6 = x2 \oplus x3 \oplus x4$$

$$c7 = x1 \oplus x2 \oplus x4$$

7 filas × 3 columnas

$$H = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad I_3$$

$$GH = 0$$

$$d_{\min} = 3 \quad t = 1 \quad R_c = 4/7$$

El síndrome: el residuo que delata el error

- R es el **vector fila recibido**, de tamaño $1 \times n$. Contiene los bits que llegaron, incluyendo posibles errores.
- El síndrome S es el vector de resultados de las comprobaciones: cada componente dice si una paridad se cumple (0) o falla (1).

$$S = RH = (C \oplus E)H = CH \oplus EH = EH$$

- El síndrome no depende de la palabra enviada, solo del patrón de error.
- $S = 0$ significa "no se detectó error", no garantiza ausencia de error.
- Un síndrome no nulo puede corresponder a **varios** patrones, el decodificador elige uno.
- Esa independencia convierte la decodificación en una tabla de 2^{n-k} entradas, no de 2^n .
- Para el Hamming (7,4): 8 entradas en lugar de 128.

El síndrome del ejemplo, sin salto matricial

- Para $R = 1101110$, las columnas de H indican qué bits sumar:

$$S_1 = r_1 \oplus r_2 \oplus r_3 \oplus r_5 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$S_2 = r_2 \oplus r_3 \oplus r_4 \oplus r_6 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

$$S_3 = r_1 \oplus r_2 \oplus r_4 \oplus r_7 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$

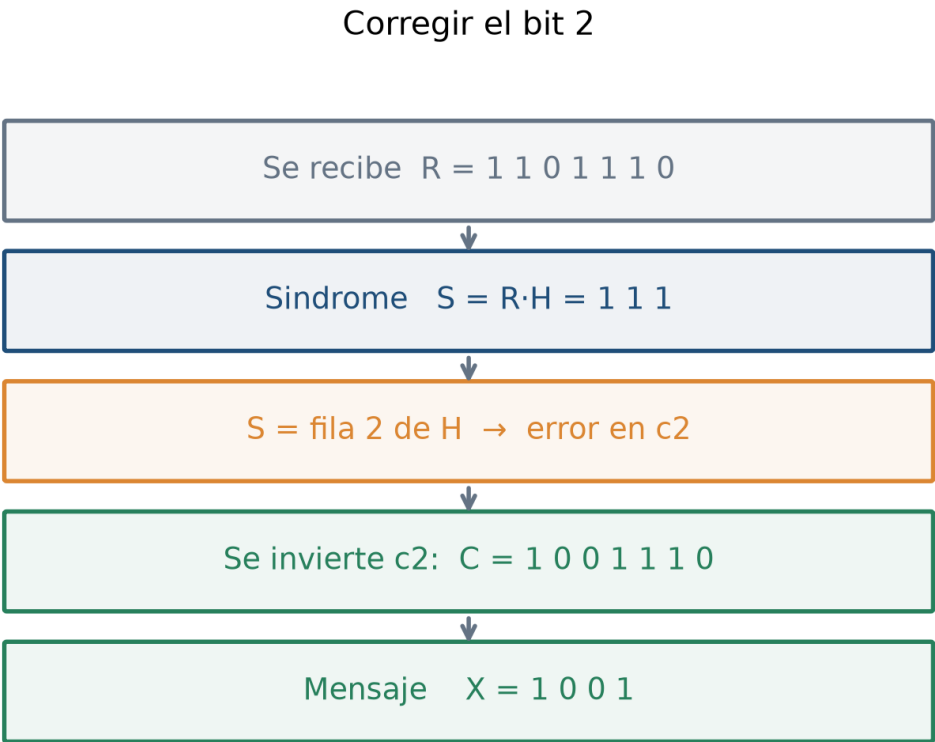
- Sale 111. No se interpreta como «bit 7»: se busca **la fila 2 de esta H**.

Tabla de síndromes y ejemplo resuelto

S = R·H: suponiendo como maximo un error

bit erroneo	sindrome S
no detectado	0 0 0
c1	1 0 1
c2	1 1 1
c3	1 1 0
c4	0 1 1
c5	1 0 0
c6	0 1 0
c7	0 0 1

7 sindromes no nulos y distintos
para 7 posiciones



Un solo producto matricial basta para localizar y reparar
Se calcula $S = RH$, se busca en qué fila de H coincide y se invierte ese bit.

Haykin, Tabla 10.4 y Ejemplo 10.4, figura original

Ejemplo 10.4 paso a paso

- Llega $R = 1101110$, el enunciado supone **un error**.

$$S = RH = 1\ 1\ 1$$

- 111 es la **fila 2** de H : el error está en c_2 .
- Se invierte ese bit: $\hat{C} = 1001110$, que cumple las paridades.
- Por ser sistemático, el mensaje es simplemente $X = 1001$.

Por qué el (7,4) corrige un error

- Si el error es único en la posición m , entonces E tiene un solo uno y:

$$S = EH = \text{fila } m \text{ de } H$$

- Las siete filas de H son distintas, así que **el síndrome nombra el bit culpable**.

$$\begin{aligned}d_{\min} &= 3 \\t &= 1 \\d_{\min} - 1 &= 2\end{aligned}$$

- Corrige uno o detecta dos: no son garantías simultáneas del corrector.

Las tres ecuaciones que gobiernan todo el tema

$$C = XG$$

$$R = C \oplus E$$

$$S = RH = EH$$

- Toda la aritmética es **módulo 2**: la suma es el OR exclusivo y cada elemento es su propio opuesto.
- Que S dependa solo de E es el motivo de que la decodificación por síndrome funcione.

Dos errores pueden parecer uno

- Dos errores, en las posiciones 1 y 2, producen:

$$E = 1100000 \quad S = 101 \oplus 111 = 010$$

- 010 también es la fila 6 de H: el corrector automático invierte el bit 6.
- Si se envió 0000000, termina en **1100010**, otra palabra válida pero equivocada.
- El síndrome detecta los dos errores, pero **no distingue** ese caso de un error simple en la posición 6.

Síndrome cero no significa canal sin errores

$$E = 1011000$$
$$EH = 0$$

- Este patrón de peso 3 también es una palabra del código.
- Al sumarlo a cualquier palabra válida se obtiene otra palabra válida.
- El receptor no puede advertir el cambio: **error no detectado**.

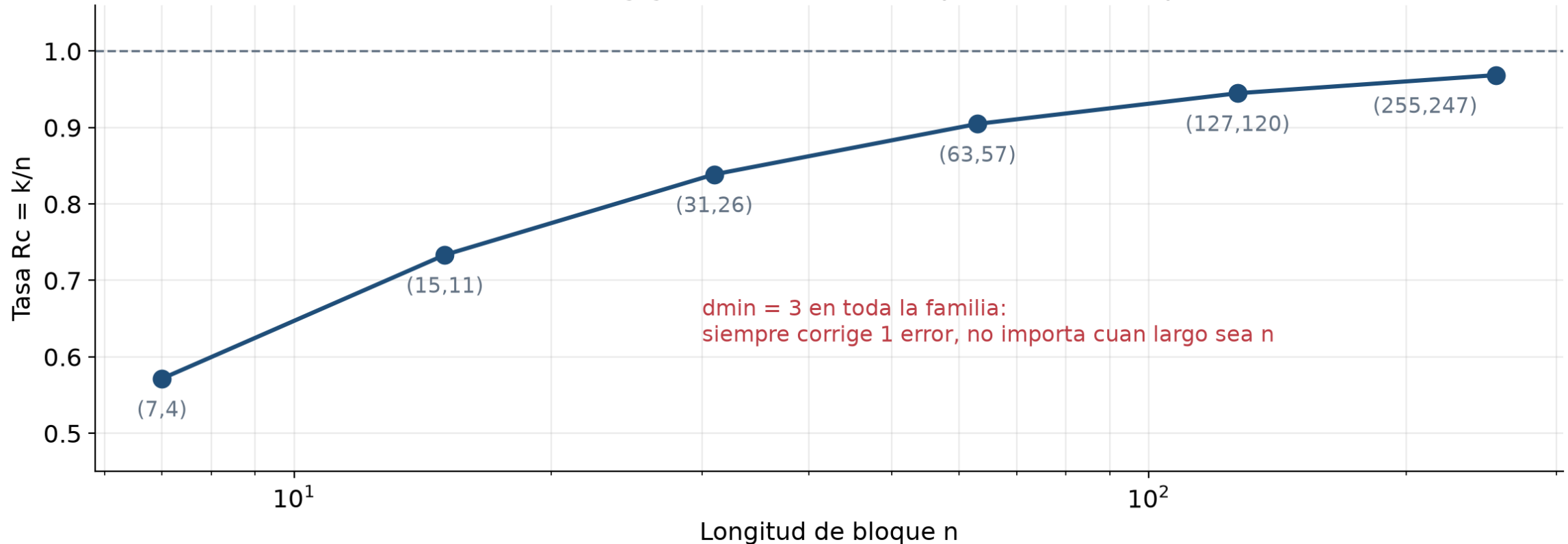
Cómo se construye la familia Hamming

$$n = 2^m - 1, \quad k = 2^m - m - 1, \quad n - k = m, \quad d_{\min} = 3$$

- Con m bits de paridad hay $2^m - 1$ síndromes no nulos.
- Se necesita **uno por cada posición de bit**: de ahí $n = 2^m - 1$.
- Es un código **perfecto**: las esferas de radio 1 llenan el espacio sin dejar huecos.
- Con $m = 3$ sale el (7,4), el ejemplo canónico del tema.
- En la gráfica de la familia, una **escala logarítmica** representa factores iguales de longitud con separaciones iguales.

La familia completa y su límite

La familia Hamming gana tasa al crecer, pero $d_{\min}$ siempre vale 3



Alargar el bloque mejora la tasa pero no la capacidad de corrección

El (63,57) tiene $R_c = 0,90$, pero sigue corrigiendo **un solo error** en 63 bits. Para más hacen falta otras familias.

Energía de bit: fórmula y ejemplo antes de comparar

- P_{av} es potencia media, en Watts.
- R_{info} es throughput de información, en bit/s.
- E_b es la **energía por bit de información**. Sin código y con un bit por símbolo, el intervalo es $T_b = 1/R_{info}$.

$$E_b = P_{av} T_b = \frac{P_{av}}{R_{info}}$$

- Potencia es energía por segundo. **Energía de bit** es lo que corresponde a cada bit útil. Un ejemplo:

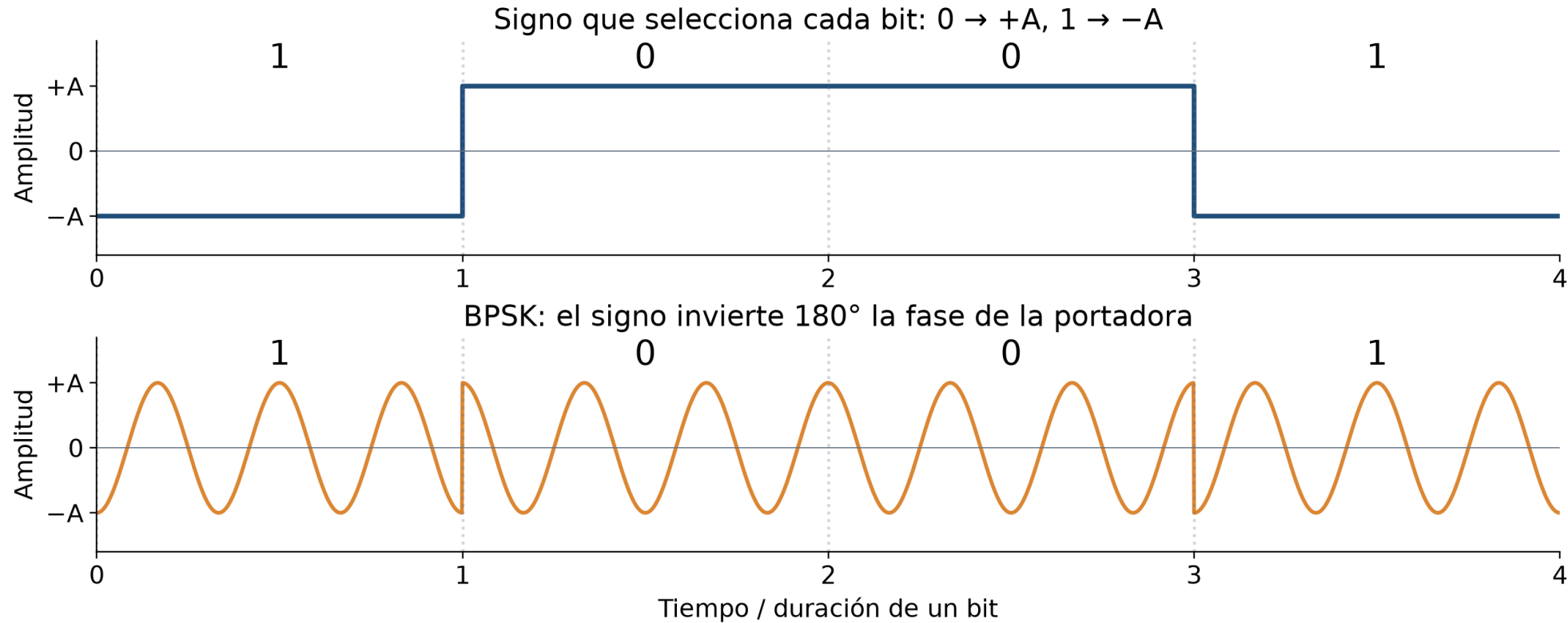
$$P_{av} = 1 \text{ mW} \quad R_{info} = 10^6 \text{ bit/s} \quad T_b = 1 \text{ } \mu\text{s}$$

$$E_b = \frac{10^{-3}}{10^6} = 10^{-9} \text{ J} = 1 \text{ nJ}$$

Antes de las curvas: señales, energía y ruido

- **BPSK** transmite 0 y 1 con fases opuestas (180°) de una portadora: **un bit por símbolo**. Usaremos 0 positivo y 1 negativo.
- Una **decisión dura** conserva solo el bit decidido a partir de la muestra, no conserva cuánto se alejaba del umbral.
- E_b : energía por **bit de información**
- E_c : energía por **bit de canal o símbolo BPSK**, ambas en julios.
- **AWGN**: ruido aditivo (se suma a la señal), blanco (densidad espectral constante en la banda del modelo) y gaussiano (amplitudes con distribución normal).
- N_0 mide la densidad espectral unilateral de potencia de ruido, en W/Hz, la bilateral es $N_0/2$. E_b/N_0 compara energía de información con ruido.
- Los **decibelios (dB)** expresan ese cociente en escala logarítmica: $10\log_{10}(E_b/N_0)$. Una diferencia en dB compara cocientes, no resta energías en julios.

BPSK: del bit al signo y a la fase



El bit cambia el signo de la portadora, no su frecuencia

Un 0 selecciona $+A$ y un 1 selecciona $-A$. La señal inferior es la portadora multiplicada por ese signo. Un cambio de signo equivale a invertir 180° la fase.

La trampa de la energía por símbolo

$$E_c = R_c E_b$$

- A igual potencia y **igual throughput**, hay que emitir n/k símbolos en el mismo tiempo.
- Cada símbolo recibe menos energía: para el (7,4), $E_c = (4/7)E_b$.
- El ruido no aumenta: baja la **energía por bit de canal** y empeora la decisión “Dura”.
- Menos energía por bit causa mayor predisposición a errores por tanto la corrección debe compensarlo.
- En el ejemplo de 1 nJ por bit útil, cada bit codificado recibe $E_c = (4/7) \cdot 1 = 0,5714$ nJ.
- En el diagrama temporal, **T es la duración de un símbolo sin codificar**, al mantener el throughput, el símbolo codificado dura T por la tasa de código.

Antes de dibujar una curva: qué error contamos

- **BER** (*Bit Error Rate*) es la fracción de bits erróneos, un **error de bloque** ocurre si el bloque no coincide con el que debía recuperarse.

Símbolo

- P_e
- P'_e
- P_b^{sin}
- P_b^{cod}

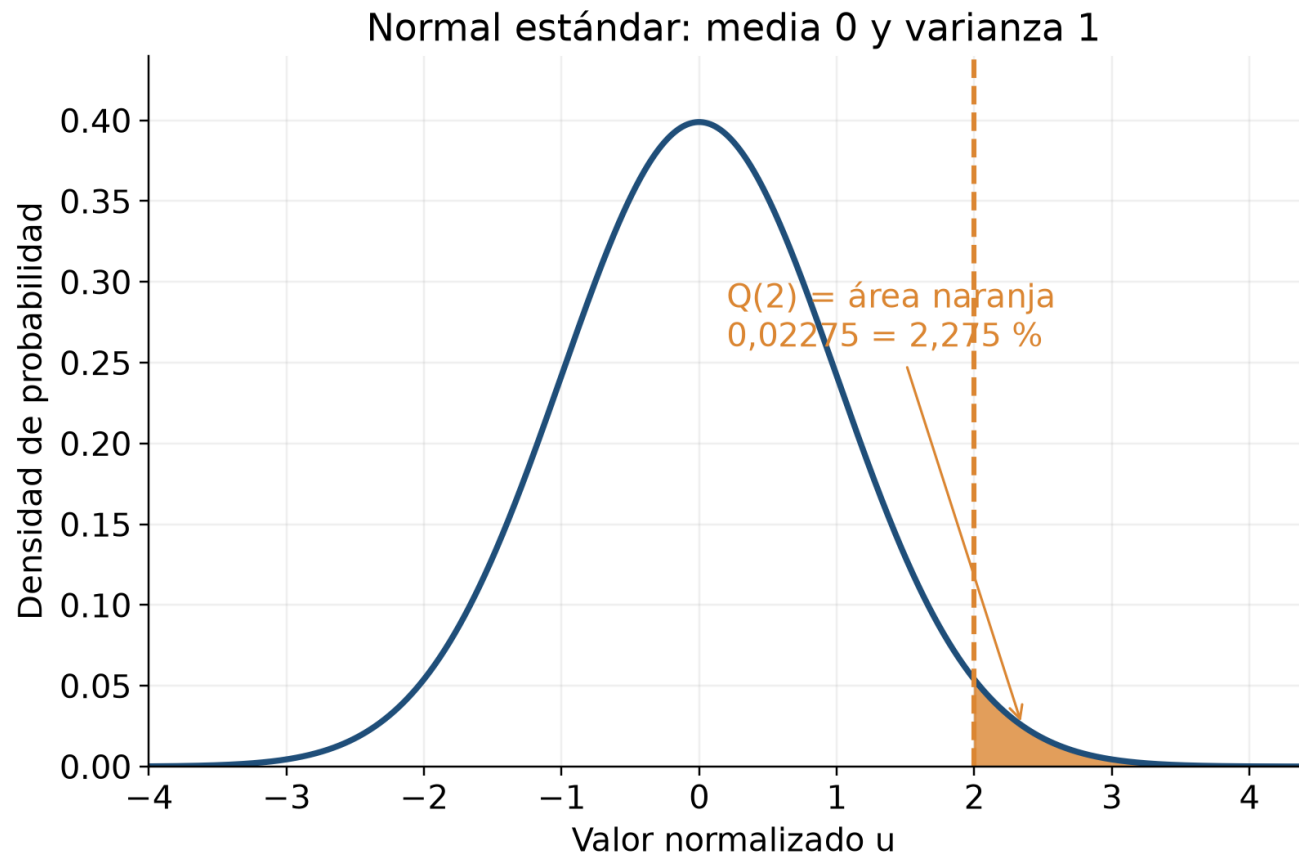
Significado

- Probabilidad de error de un bit BPSK sin código
- Probabilidad de error de un bit BPSK con código de Hamming
- Probabilidad de error de **bloque (1 o más errores) sin código**
- Probabilidad de error de **bloque (2 o más errores) con código**

- La BER después de decodificar requiere contar los bits de información que quedan mal.
- Compararemos primero **bloques de siete bits**, para comparar el mismo tamaño de bloque.

Q: el área que representa la probabilidad de error

- $Q(z)$ es el **área a la derecha de z** en una normal estándar, con media 0 y varianza 1.



Cómo usar el argumento z

$$Q(1) = 0,158655$$

$$Q(2) = 0,022750$$

$$Q(3) = 0,001350$$

Mayor z
→ menor área de error

La altura NO es la probabilidad.
La probabilidad es el área.

Q: el área que representa la probabilidad de error

Table 1: Values of $Q(x)$ for $0 \leq x \leq 9$

x	$Q(x)$	x	$Q(x)$	x	$Q(x)$	x	$Q(x)$
0.00	0.5	2.30	0.010724	4.55	2.6823×10^{-6}	6.80	5.231×10^{-12}
0.05	0.48006	2.35	0.0093867	4.60	2.1125×10^{-6}	6.85	3.6925×10^{-12}
0.10	0.46017	2.40	0.0081975	4.65	1.6597×10^{-6}	6.90	2.6001×10^{-12}
0.15	0.44038	2.45	0.0071428	4.70	1.3008×10^{-6}	6.95	1.8264×10^{-12}
0.20	0.42074	2.50	0.0062097	4.75	1.0171×10^{-6}	7.00	1.2798×10^{-12}
0.25	0.40129	2.55	0.0053861	4.80	7.9333×10^{-7}	7.05	8.9459×10^{-13}
0.30	0.38209	2.60	0.0046612	4.85	6.1731×10^{-7}	7.10	6.2378×10^{-13}
0.35	0.36317	2.65	0.0040246	4.90	4.7918×10^{-7}	7.15	4.3389×10^{-13}
0.40	0.34458	2.70	0.003467	4.95	3.7107×10^{-7}	7.20	3.0106×10^{-13}
0.45	0.32636	2.75	0.0029798	5.00	2.8665×10^{-7}	7.25	2.0839×10^{-13}
0.50	0.30854	2.80	0.0025551	5.05	2.2091×10^{-7}	7.30	1.4388×10^{-13}
0.55	0.29116	2.85	0.002186	5.10	1.6983×10^{-7}	7.35	9.9103×10^{-14}
0.60	0.27425	2.90	0.0018658	5.15	1.3024×10^{-7}	7.40	6.8092×10^{-14}
0.65	0.25785	2.95	0.0015889	5.20	9.9644×10^{-8}	7.45	4.667×10^{-14}
0.70	0.24196	3.00	0.0013499	5.25	7.605×10^{-8}	7.50	3.1909×10^{-14}
0.75	0.22663	3.05	0.0011442	5.30	5.7901×10^{-8}	7.55	2.1763×10^{-14}
0.80	0.21186	3.10	0.0009676	5.35	4.3977×10^{-8}	7.60	1.4807×10^{-14}
0.85	0.19766	3.15	0.00081635	5.40	3.332×10^{-8}	7.65	1.0049×10^{-14}
0.90	0.18406	3.20	0.00068714	5.45	2.5185×10^{-8}	7.70	6.8033×10^{-15}
0.95	0.17106	3.25	0.00057703	5.50	1.899×10^{-8}	7.75	4.5946×10^{-15}
1.00	0.15866	3.30	0.00048342	5.55	1.4283×10^{-8}	7.80	3.0954×10^{-15}

El modelo: BPSK, ruido gaussiano y decisión dura

- Primero calculamos el argumento de Q. Después buscamos el área correspondiente.
- **Detección coherente:** el receptor conoce la fase de referencia. **Bits equiprobables:** 0 y 1 tienen igual probabilidad.

$$P_e = Q\left(\sqrt{\frac{2E_b}{N_0}}\right)$$
$$P'_e = Q\left(\sqrt{\frac{2E_c}{N_0}}\right) = Q\left(\sqrt{\frac{8E_b}{7N_0}}\right)$$

- Se suponen errores independientes, detección coherente y bits equiprobables.
- A igual E_b/N_0 , $P'_e > P_e$: todavía no hemos aplicado la corrección.
- Ejemplo: $E_b/N_0 = 2$ en escala lineal da $\sqrt{2 \cdot 2} = 2$. Entonces $P_e = Q(2) = 0,02275$.

Ejemplo 10.5: contar los bloques que fallan

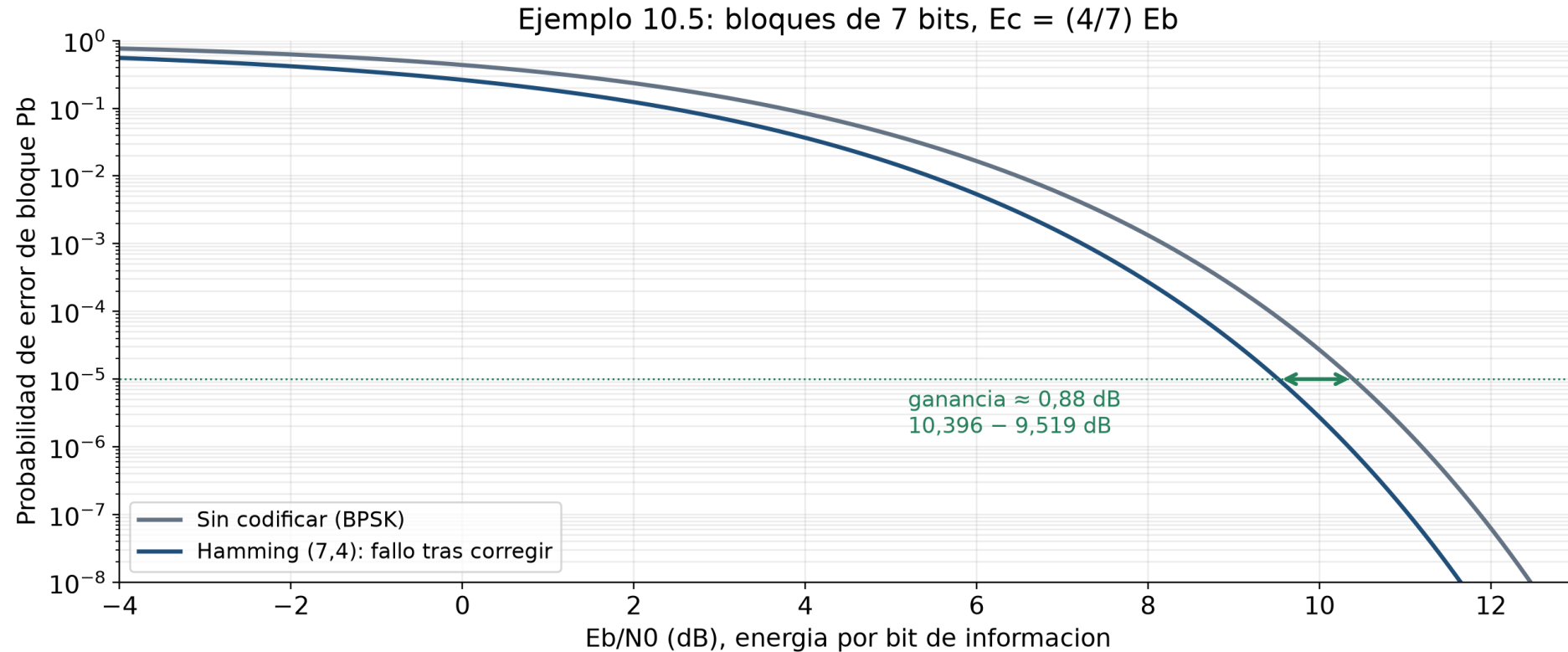
$$P_b^{\text{sin}} = 1 - (1 - P_e)^7$$

- Sin código: falla si hay **uno o más** errores.

$$P_b^{\text{cod}} = 1 - (1 - P'_e)^7 - 7P'_e(1 - P'_e)^6$$

- Con Hamming: restamos los casos de **cero y un error**, que sí se recuperan.
- Los bloques tienen igual longitud transmitida, transportan 7 y 4 bits de información, respectivamente.

El balance real de la apuesta



A error de bloque 10^{-5} : aproximadamente 0,88 dB de ganancia

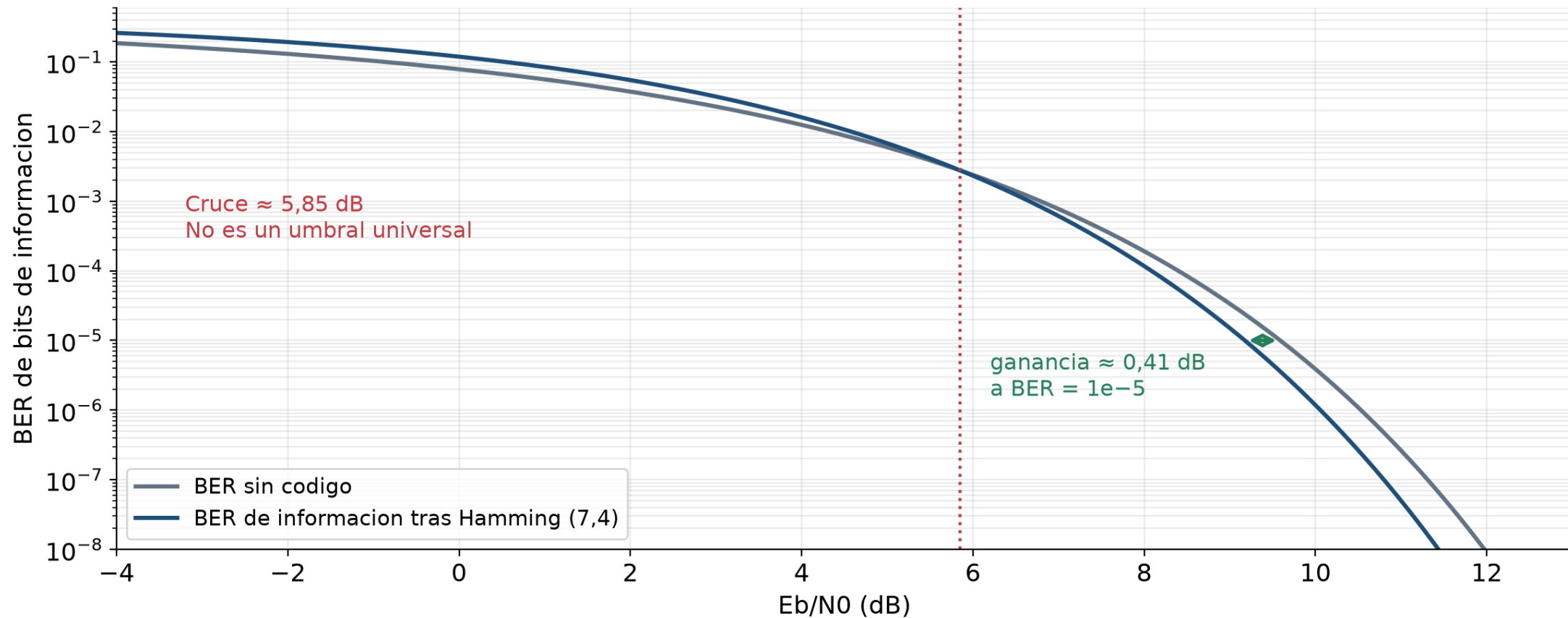
Leemos horizontalmente: 10,396 dB sin código frente a 9,519 dB con código. La diferencia es 0,877 dB para este modelo y esta métrica, no una ganancia universal.

Una cuenta completa a 6 dB

Paso	Resultado
• Pasar a escala lineal • Error de bit sin código • Error de bloque sin código • Error de bloque con Hamming	• $E_b/N_0 = 10^{6/10} = 3,981$ • $P_e = 0,002388$ • $P_b^{\text{sin}} = 0,01660$ • $P_b^{\text{cod}} = 0,005386$
• Baja la probabilidad de bloque incorrecto aunque cada bit de canal tenga menos energía.	

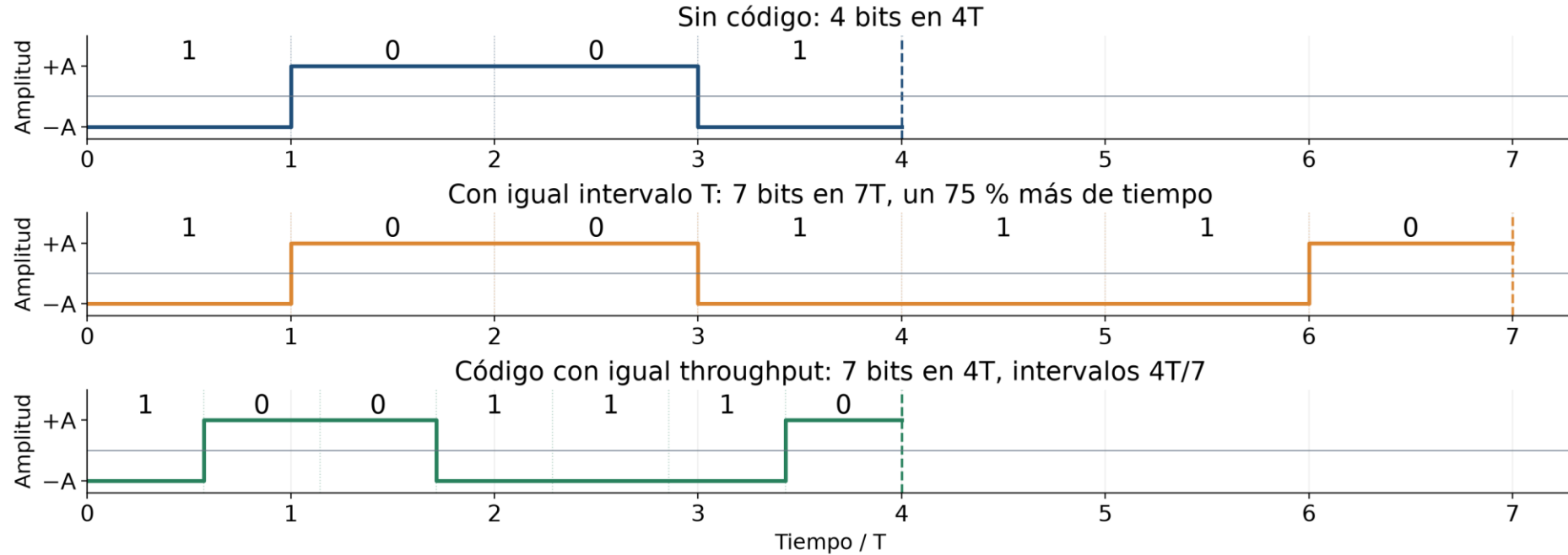
Si preguntamos por BER, cambia la comparación

BPSK + AWGN + corrector de un error



Enumerando los 128 patrones de error: cruce a 5,85 dB, a BER 10^{-5} , $9,588 \rightarrow 9,174$ dB, ganancia de 0,41 dB. BER y error de bloque son métricas distintas.

El otro precio: el ancho de banda



$$T_c = R_c T$$

$$B_{\text{cod}} = \frac{B_{\text{sin}}}{R_c} = \frac{7}{4} B_{\text{sin}}$$

Si dejamos T fijo, el mensaje tarda 7T en lugar de 4T. Si conservamos el throughput, hay que acortar los intervalos y aumentar la banda.

Las tres monedas del diseñador

Recurso

- Potencia
- Ancho de banda
- Redundancia (R_c)

Beneficio y coste

- Menos BER, más energía, interferencia y coste
- Más throughput, más espectro ocupado
- Posible corrección, banda o throughput, memoria y cálculo

- La codificación de canal **cambia potencia por ancho de banda**.
- No hay comida gratis: solo la elección de qué recurso sobra en cada sistema.

Qué mantenemos fijo al comparar

Restricción

- Mismo throughput y potencia
- Misma banda y modulación

Efecto de usar Hamming (7,4)

- 7/4 veces más símbolos, energía por símbolo 4/7
- Igual velocidad de símbolos, throughput multiplicado por 4/7

- No se pueden conservar simultáneamente potencia, banda y throughput y exigir una mejora gratuita.
- La comparación usa energía **por bit de información**. No comparar E_c con E_b como si fueran iguales.

La distancia euclíde

- d_E es la **distancia euclídea entre señales**: la raíz de la suma de diferencias de sus coordenadas al cuadrado. Cada coordenada corresponde aquí a un símbolo.

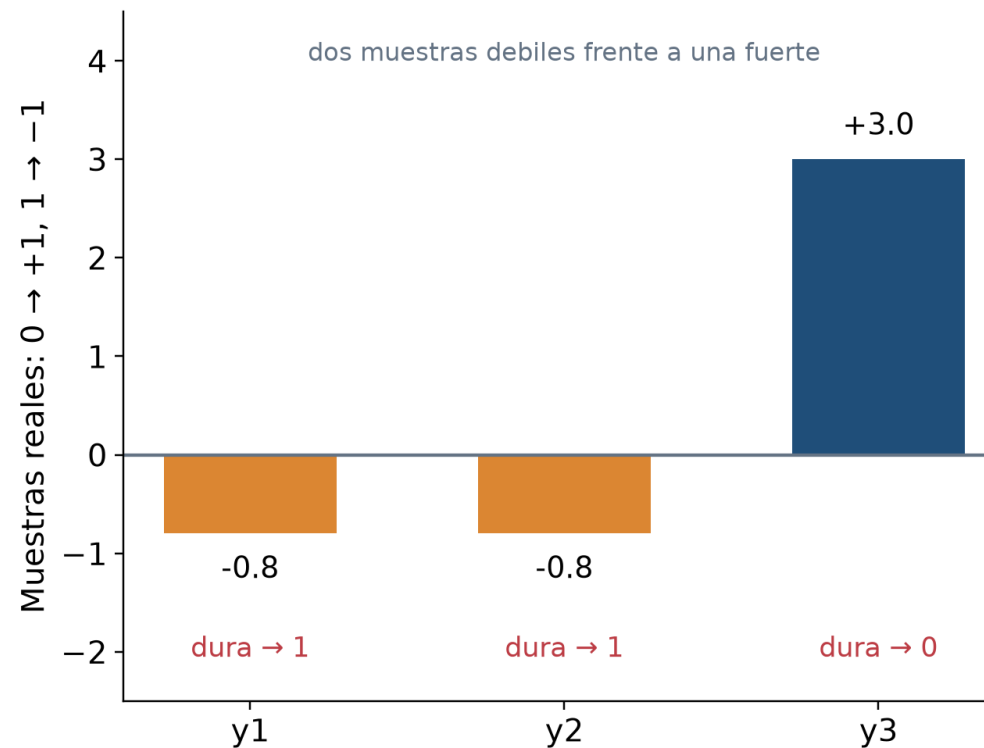
$$d_E = 2 \sqrt{E_c d_H(C_i, C_j) E_c}$$

Lo que el demodulador tira a la basura

- El demodulador recibe un **valor real** por cada símbolo, no un bit.
- La **decisión dura** lo redondea a 0 o 1 antes de pasarlo al decodificador.
- Con la convención 0 positivo y 1 negativo, +0,1 y +3,0 se vuelven el mismo "0": se pierde **confianza**.
- La **decisión blanda** entrega el valor real y deja que el decodificador use esa confianza.

Las cuentas del ejemplo

- **Repetición (3,1)** envía tres copias de un bit: solo son válidas 000 y 111. Usamos $0 \mapsto +1$, $1 \mapsto -1$.
- Muestras $y = (-0,8, -0,8, 3)$. La decisión dura da $R = 110$, que se corrige a 111 por mayoría.



Las cuentas del ejemplo

- **Repetición (3,1)** envía tres copias de un bit: solo son válidas 000 y 111. Usamos $0 \mapsto +1$, $1 \mapsto -1$.
- Muestras $y = (-0,8, -0,8, 3)$. La decisión dura da $R = 110$, que se corrige a 111 por mayoría.
- D_{111} y D_{000} son las distancias euclídeas de esas muestras a las señales de 111 y 000:

$$D_{111} = \sqrt{[-0,8 - (-1)]^2 + [-0,8 - (-1)]^2 + [3 - (-1)]^2}$$

$$D_{111} = \sqrt{0,2^2 + 0,2^2 + 4^2} = \sqrt{16,08} = 4,0100$$

- Restamos cada muestra a la amplitud que produciría 111, es decir, **(-1, -1, -1)**.

Las restas completas para la otra palabra

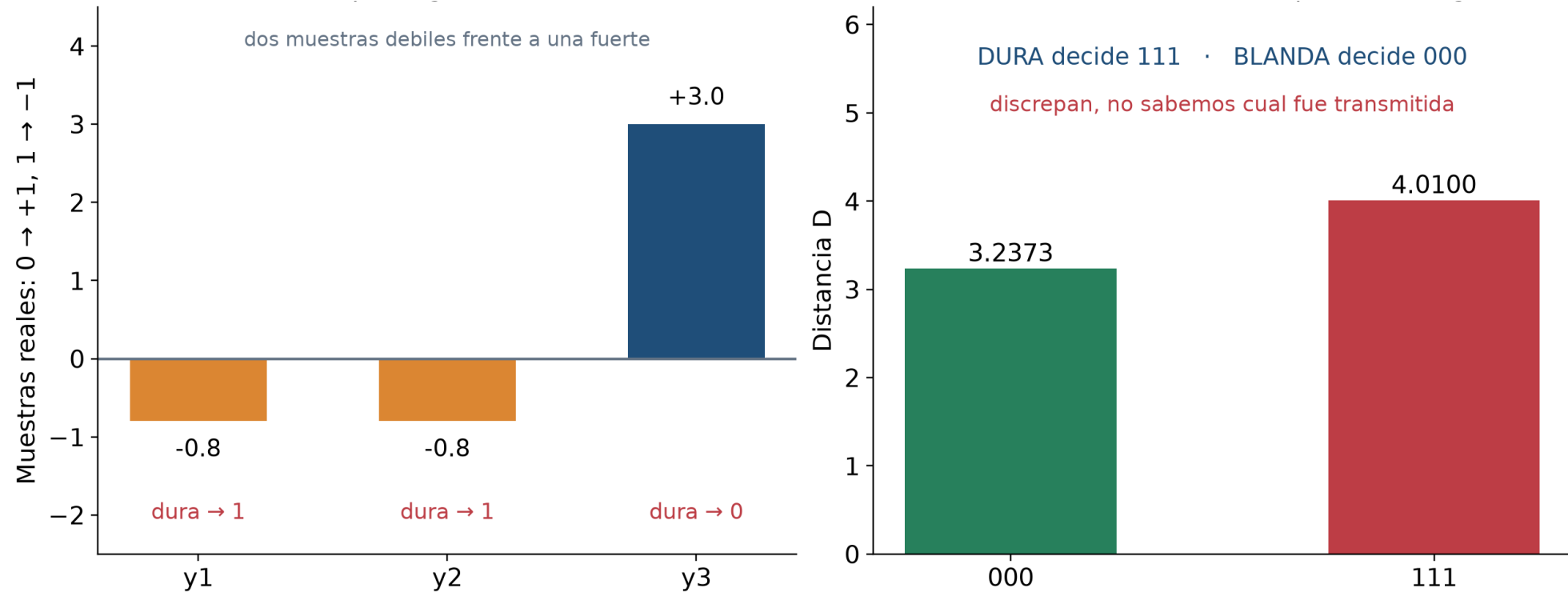
- La palabra 000 produciría **(+1, +1, +1)**. Se usan las mismas muestras, sin redondearlas.

$$D_{000} = \sqrt{[-0,8 - (+1)]^2 + [-0,8 - (+1)]^2 + [3 - (+1)]^2}$$

$$D_{000} = \sqrt{(-1,8)^2 + (-1,8)^2 + 2^2} = \sqrt{10,48} = 3,2373$$

- Como 3,2373 es menor que 4,0100, la decisión blanda selecciona **000**.
- La decisión dura seleccionó 111. No conocemos lo transmitido, por lo que este caso demuestra una discrepancia, no un acierto garantizado.

El ejemplo que las enfrenta



Las dos decisiones discrepan: la blanda conserva la confianza

Los dos primeros valores son débiles, el tercero es fuerte. Sin conocer lo transmitido no podemos afirmar cuál acertó en este caso.

Por qué la decisión blanda gana

- Con los signos adoptados, $+3,0$ pesa más que los dos $-0,8$ juntos.
- La decisión dura le da a los tres el mismo peso: un voto cada uno.
- La blanda pondera por confianza y deja que la evidencia fuerte mande.
- Bajo AWGN, elegir distancia mínima es **máxima verosimilitud**: escoger la palabra que hace más probable observar esas muestras. Con palabras equiprobables también minimiza el error.
- Mejora estadísticamente al conservar información, no garantiza acierto en cada bloque ni una ganancia fija de 2–3 dB.

Más allá de Hamming

Familia	Idea y aplicaciones de contexto
• Reed-Solomon • BCH • Convolucionales • Turbo / LDPC	• Grupos de bits, ráfagas (errores consecutivos) que afectan pocos grupos. CD, DVD, QR, espacio • Capacidad diseñada dentro de límites de longitud y tasa. Almacenamiento • Trellis: grafo de estados de la memoria. Viterbi busca la ruta más probable. GSM, satélite y módems • Iteran: repiten y refinan cálculos. LDPC: paridad de baja densidad (H con pocos unos). Turbo: LTE, LDPC: datos 5G, DVB-S2, Wi-Fi
• Comparten tasa, distancia y complejidad, no todos usan la misma decodificación por síndrome.	

Convolutionales: la paridad recuerda el pasado

- j es el instante de entrada, x_j es el bit actual y p_j la paridad calculada. Los índices anteriores representan bits guardados en memoria.

$$p_j = x_j \oplus x_{j-2} \oplus x_{j-3} \oplus x_{j-5}$$

- Por cada entrada x_j se transmiten x_j y p_j : tasa nominal 1/2.
- La memoria alcanza cinco posiciones anteriores, no se reinicia con cada bit.
- En un no sistemático se transmiten dos paridades, no necesariamente el bit original.
- Más memoria puede proteger mejor, pero aumenta el coste del trellis y de Viterbi.

Turbo: dos perspectivas del mismo mensaje

- **Intercalar** reordena los mismos bits mediante una permutación conocida, no añade información. p y q son las dos secuencias de paridad.
- x_j^i representa el mensaje **intercalado**, la i no es una potencia.

$$p_j = p_{j-1} \oplus x_j \oplus x_{j-2} \oplus x_{j-4}$$

$$q_j = q_{j-1} \oplus x_j^i \oplus x_{j-2}^i \oplus x_{j-4}^i$$

- Se envían x_j, p_j, q_j : tasa nominal 1/3, sin contar terminación (bits para dejar los codificadores en un estado conocido).
- Son **recursivas** porque dependen de una paridad anterior, aportan evidencias que el receptor combina iterativamente.

Vocabulario para seguir los diagramas ARQ

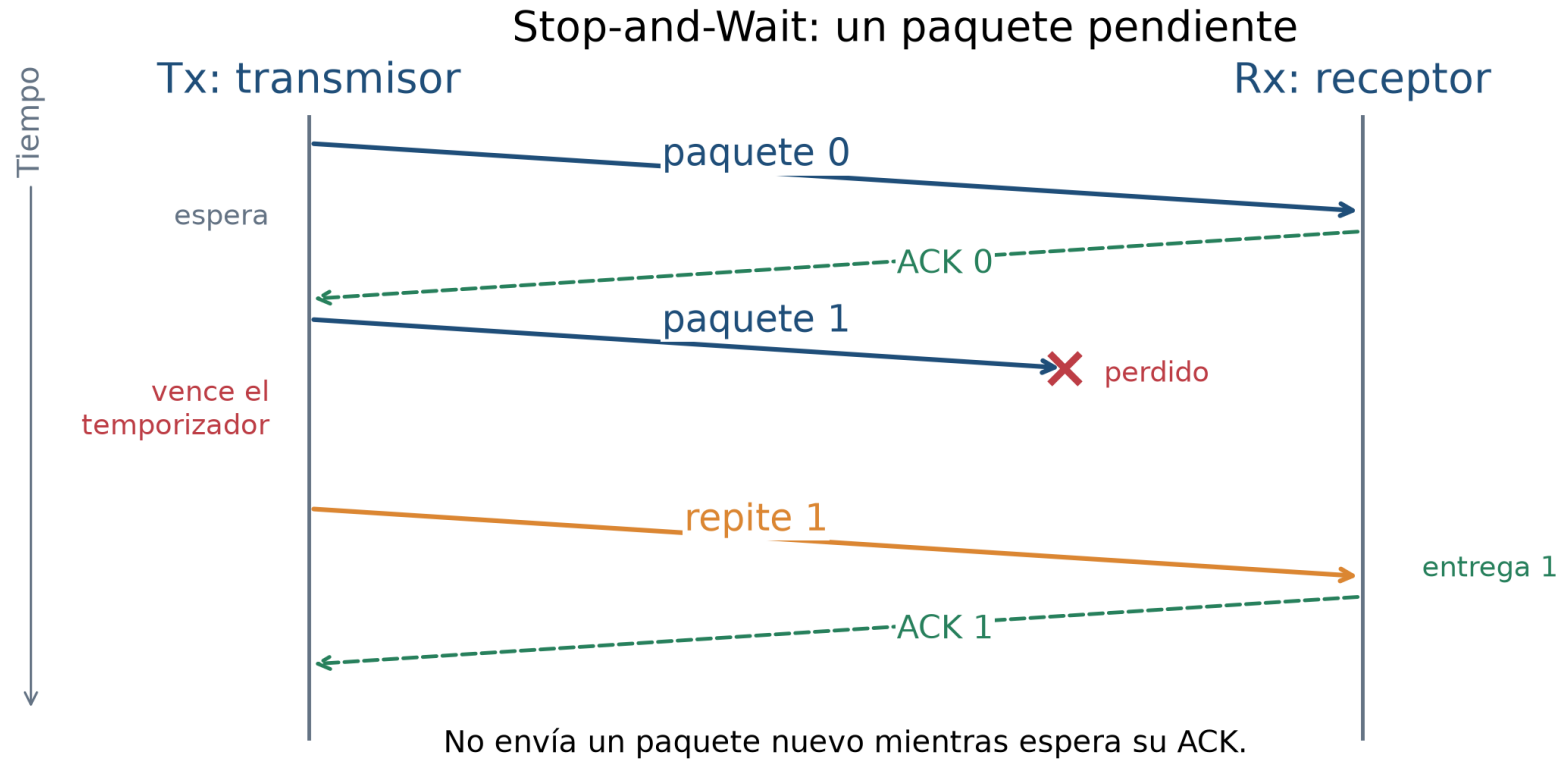
- Un **paquete** agrupa bits de datos y control. **Full-duplex** permite enviar y recibir a la vez. Una ventana permite tener varios paquetes pendientes.
- **ACK** confirma recepción, **NAK** pide repetir. Viajan por el canal de retorno simultáneo.
- Un **búfer** es memoria temporal para guardar paquetes. Un **temporizador** permite reintentar cuando no llega la confirmación.
- **Ventana**: máximo de paquetes sin ACK, numeración y temporizadores gestionan ACK perdidos.

ARQ Stop-and-Wait: enviar uno y esperar

- El transmisor envía un paquete y **se detiene** hasta recibir el reconocimiento.
- Funciona en **half-duplex** (cada sentido transmite por turnos), con hardware mucho más simple.
- El precio es el tiempo muerto de ida y vuelta, que puede dominar el enlace.
- T_P : transmitir el paquete, T_T : propagación de ida y vuelta, T_D : ACK más retardos. T es el ciclo completo:

$$T = T_P + T_T + T_D$$

Stop-and-Wait: qué ocurre cuando se pierde un paquete



No se envía un paquete nuevo hasta cerrar el anterior

Las flechas sólidas llevan datos. Las discontinuas llevan ACK. Si vence el temporizador, se repite el paquete pendiente.

Problema resuelto: cuánto cuesta esperar

- Paquetes de **210 bits** a **100 kbit/s**, distancia **300 km**, ACK de 10^{-4} s.
- R_{ef} cuenta bits del paquete por segundo, **carga útil** excluye los bits añadidos para control y protección.
- En la figura, **Tx/Rx** indican transmisor/receptor, **latencia** es el tiempo hasta disponer de los datos enviados.

$$T_P = 210 \cdot 10^{-5} = 2,10 \text{ ms}$$
$$T_T = \frac{2 \cdot 300 \cdot 10^3}{3 \cdot 10^8} = 2,00 \text{ ms}$$
$$T_D = 0,10 \text{ ms}$$

$$T = 4,20 \text{ ms}$$

$$R_{\text{ef}} = \frac{210}{4,20 \text{ ms}} = 50\,000 \text{ bit/s}$$

- **La tasa efectiva cae a la mitad** de los 100 kbit/s nominales, y aún no hubo ningún error.
- Los 210 bits incluyen protección: esta no es todavía la tasa de **carga útil**.

Y ahora con errores: la serie de retransmisiones

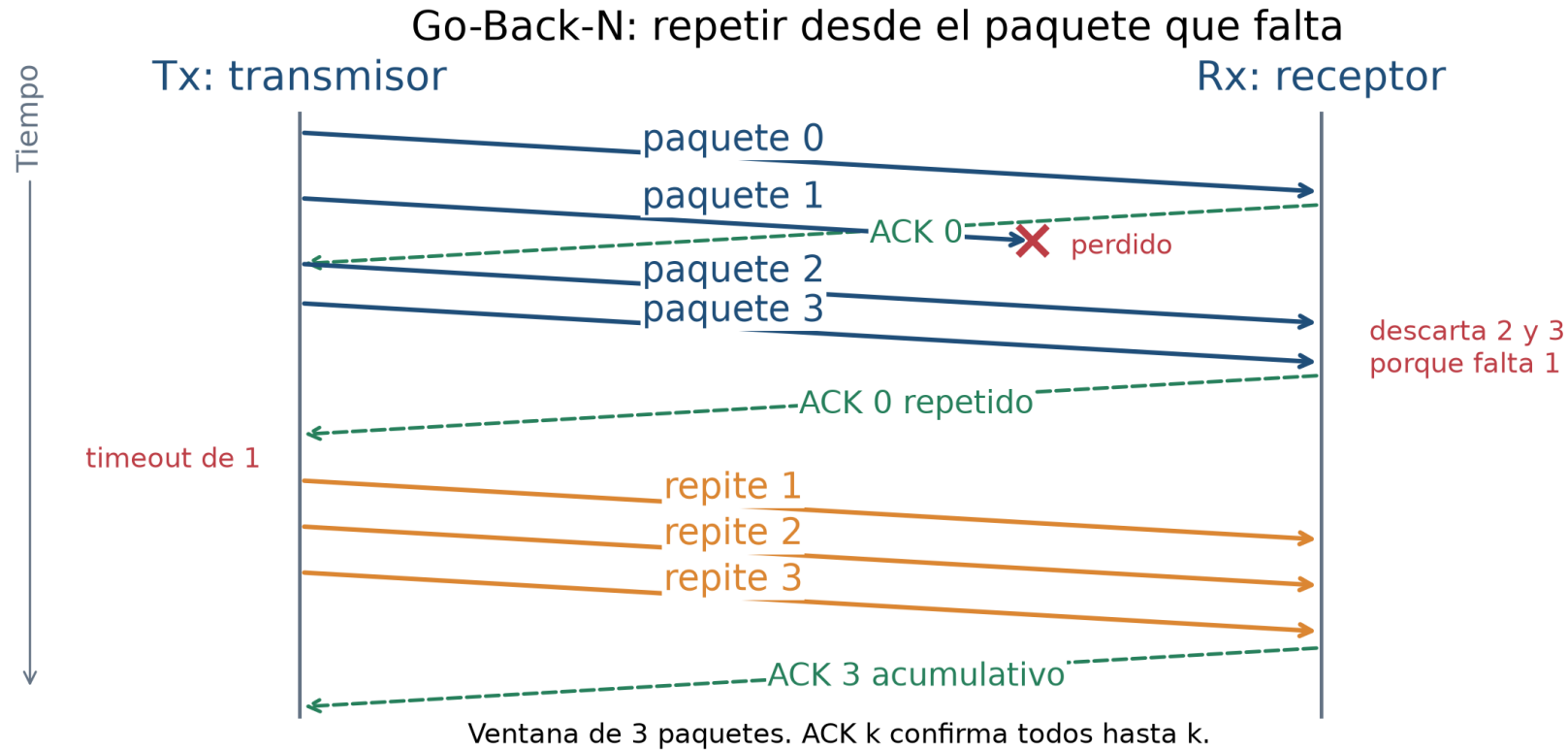
- T_m es el **tiempo medio hasta completar un envío**, j cuenta reintentos. Cada intento adicional cuesta otro ciclo T .
- Aquí p_{paq} es la probabilidad de error de **paquete**, no BER. Intentos independientes y ACK fiable.

$$T_m = T(1 - p_{\text{paq}}) + 2T p_{\text{paq}}(1 - p_{\text{paq}}) + \dots$$

$$T_m = T(1 - p_{\text{paq}}) \sum_{j=0}^{\infty} (j + 1) p_{\text{paq}}^j = \frac{T}{1 - p_{\text{paq}}}$$

- Con $p_{\text{paq}} = 10^{-2}$: $T_m = 4,242$ ms y **49 500 bit/s**.
- Cuando p_{paq} se acerca a 1, el tiempo medio crece sin límite.

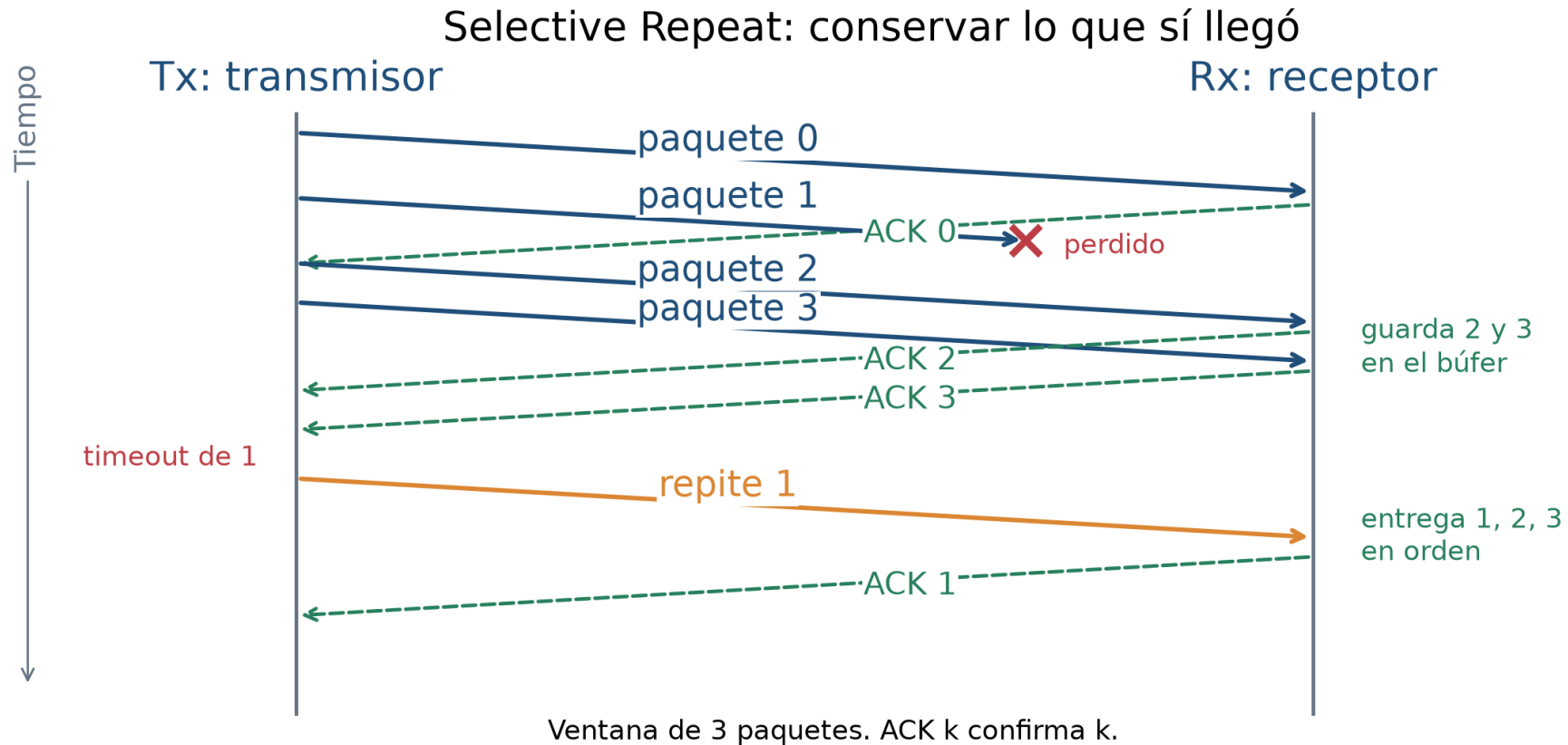
Go-Back-N: una pérdida hace repetir varios paquetes



El receptor entrega solo la secuencia continua

Al faltar el paquete 1, descarta los paquetes 2 y 3. El transmisor vuelve a enviar desde el primero no confirmado. Un ACK acumulativo confirma todos los paquetes hasta el número indicado.

Selective Repeat: repetir solo el paquete que falta



El búfer evita desperdiciar paquetes que llegaron bien

El receptor confirma y guarda los paquetes 2 y 3. Se retransmite solo el 1. Cuando llega, se entregan 1, 2 y 3 en orden.

Cuándo elegir cada estrategia

Criterio	Comparación
• Retorno • Complejidad • Banda • Canal • Retardo	• ARQ lo requiere, FEC no • Stop&Wait simple, FEC depende del código • ARQ paga retransmisiones, FEC paga paridad • ARQ funciona bien con pocos reintentos, FEC dentro de su capacidad • Archivos toleran espera, audio y vídeo favorecen evitarla
• Difusión sin retorno y lectura de soportes favorecen FEC. • Streaming puede tener retorno, puede usar ARQ, FEC o ambos según el plazo.	

Lo esencial en cinco ecuaciones

$$R_c = \frac{k}{n}$$

$$C = XG$$

$$S = RH = EH$$

$$t = \left\lceil \frac{d_{\min} - 1}{2} \right\rceil$$

$$E_c = R_c E_b$$

- Más protección exige redundancia, a throughput y potencia fijos, **menos** energía por bit de canal.

Las ideas que hay que llevarse

- La redundancia **estructurada** permite detectar y, si se gasta el doble de distancia, corregir.
- $d_{\min}$ lo decide todo: detecta $d_{\min} - 1$, corrige $t = \lfloor (d_{\min} - 1)/2 \rfloor$, o se reparte.
- La **linealidad** hace viable codificar (G) y decodificar (H y el síndrome) sin tablas gigantes.
- El síndrome **solo depende del error**, pero distintos errores comparten firma.
- Codificar **no siempre compensa**: por debajo del umbral de E_b/N_0 , empeora la BER.
- **ARQ** requiere retorno y tolerancia a reintentos, **FEC** no requiere retorno. Pueden combinarse.
- La decisión **blanda** conserva confianza, su ganancia depende del modelo y del código.

Una última comprobación antes de cerrar

Pregunta

- ¿S nulo garantiza transmisión correcta?
- ¿Hamming corrige uno y distingue todos los dobles?
- ¿Más paridad siempre protege mejor?
- ¿La curva del Ej. 10.5 es BER?

Respuesta razonada

- No: un error que sea palabra válida deja S nulo
- No, para esa garantía simultánea hace falta más distancia
- No, comparar con el contraejemplo 10.3
- No: es error de bloque de siete bits